

Multimodal Visual Understanding

Multimodal Visual Understanding

1. Course Content
2. Preparation
 - 2.1 Starting the Agent
3. Running the Examples
 - 3.1 Starting the Program
 - 3.2 Test Cases
 - 3.2.1 Case 1
 - 3.2.2 Case 2
4. Source Code Analysis

1. Course Content

Run the example program, allowing the robot to observe the environment through text interaction in the terminal and perform tasks based on instructions.

[!NOTE]

- The only difference between the text version and the voice version is the method of instruction input; the text version does not include speech recognition and speech synthesis playback.

2. Preparation

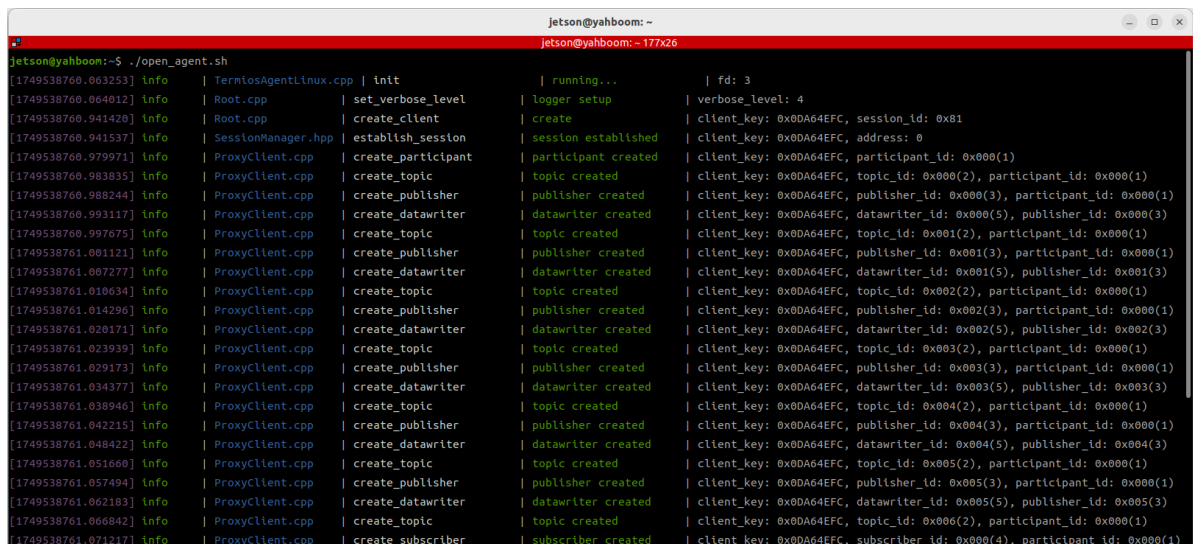
2.1 Starting the Agent

Note: The Docker agent must be started before testing all cases. If it is already running, there is no need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:



```
Jetson@yahboom:~$ ./open_agent.sh
[1749538760.063253] Info | TermiosAgentLinux.cpp | Init | running... | fd: 3
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)
[1749538761.066042] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Running the Examples

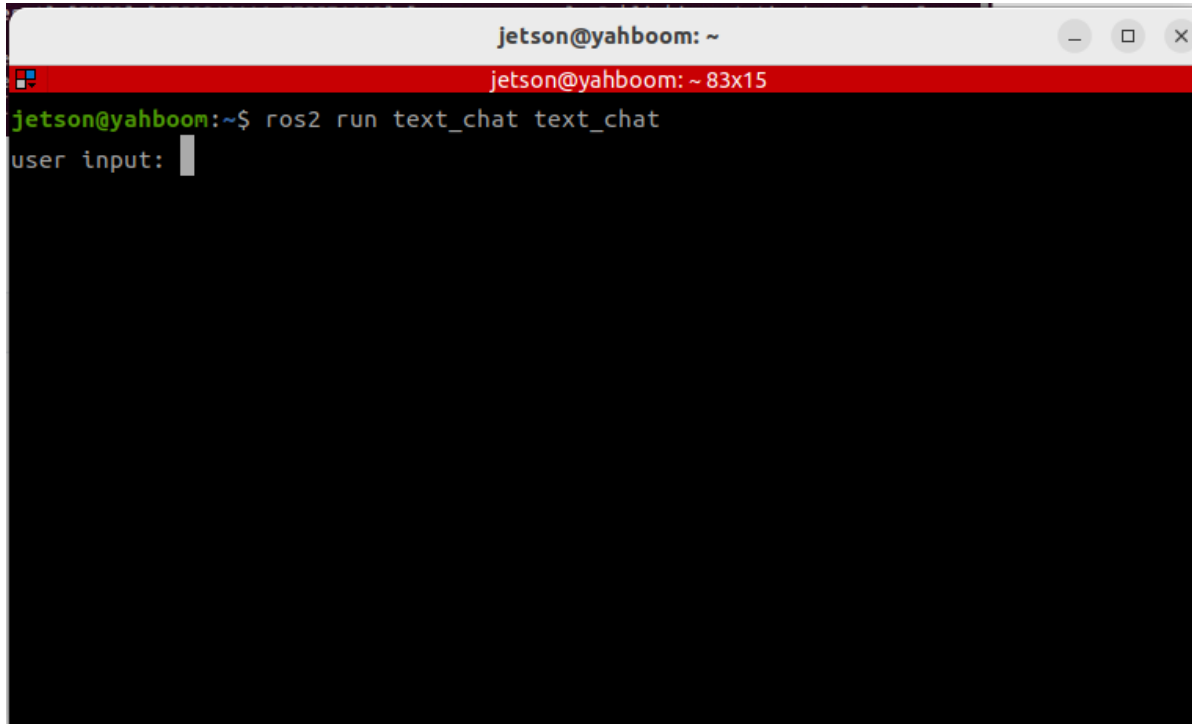
3.1 Starting the Program

- Open the terminal on the vehicle and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

- Start the text interaction node on the vehicle:

```
ros2 run text_chat text_chat
```



3.2 Test Cases

Here are two example test cases; users can create their own test instructions.

- Tell me what objects are in front of you, and describe their functions.
- Please look to see if there is a blue block and a pack of tissues in front of you. If there is, nod your head; if not, shake your head.

3.2.1 Case 1

Enter the test case in the text interaction terminal. After the model thinks, the model will reply to the user and perform the actions according to the user's instructions.

```
jetson@yahboom: ~  
jetson@yahboom: ~ 90x24  
jetson@yahboom:~$ ^C  
jetson@yahboom:~$ ros2 run text_chat text_chat  
user input: 帮我看看你面前都有什么物品，并说一说这些物品都有什么作用  
okay😊, let me think for a moment... \[INFO] [1750319399.085616757] [text_chat_node]: robot  
ot response: The upcoming task to be carried out: 1.调用视觉函数获取当前画面，观察周围环境  
，然后描述看到的物品和物品的作用  
user input: [INFO] [1750319404.156804758] [text_chat_node]: robot response: "action": ['se  
ewhat()'], "response": 好的，让我用我的大眼睛看看面前都有些什么物品吧！  
[INFO] [1750319412.358271064] [text_chat_node]: robot response: "action": [], "response":  
我看到面前的桌子上有一包纸巾、一支笔和一把螺丝刀。纸巾可以用来擦拭或者清洁，笔是用来书写和  
记录的工具，而螺丝刀则是用来拧紧或松开螺丝的，常用于组装或修理物品。这些小物件各有各的用途  
，真是生活中的小帮手呢！  
[INFO] [1750319417.010779878] [text_chat_node]: robot response: "action": ['finishtask()']  
， "response": 我已经描述完面前的物品及其作用啦，如果你还有其他需要帮忙的，随时告诉我！  
user input: 结束当前任务  
okay😊, let me think for a moment... \[INFO] [1750319434.468903279] [text_chat_node]: rob  
ot response: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了，有需要再叫  
我吧！  
user input:
```

After the robot completes a task, it enters a waiting state. At this point, you can continue the conversation. The instructions will be directly passed to the execution layer's large language model, and all conversation history will be retained. You can enter the "end current task" command again to have the robot end the current task cycle and start a new one.

3.2.2 Case 2

Similar to Case 1, entering Case 2 in the terminal will cause the model to respond and perform actions according to the instructions.

```
jetson@yahboom: ~  
jetson@yahboom: ~ 84x20  
jetson@yahboom:~$ ros2 run text_chat text_chat  
user input: 请你看看面前有没有蓝色方块和一包纸巾，有的话你就点点头，没有的话你就摇头  
摇头  
okay😊, let me think for a moment... \[INFO] [1750319643.210169487] [text_chat_node  
]: robot response: The upcoming task to be carried out: 1. 调用视觉函数获取当前画面2  
.如果画面中有蓝色方块和一包纸巾,如果有，调用机械臂点头函数，如果没有，调用机械臂摇头  
函数  
user input: [INFO] [1750319647.115646110] [text_chat_node]: robot response: "action"  
: ['seewhat()'], "response": 好的，我这就用我的火眼金睛看看面前有什么。  
[INFO] [1750319655.510956964] [text_chat_node]: robot response: "action": ['arm_shak  
e()'], "response": 我看到面前有一包纸巾和一个黄色螺丝刀，但是没有蓝色方块哦，所以我  
摇摇头。  
[INFO] [1750319665.246268606] [text_chat_node]: robot response: "action": ['finishta  
sk()'], "response": 我已经完成了检查和摇头的动作，有其他事情需要帮忙吗？
```

4. Source Code Analysis

- Robot action source code path:

```
~/M3Pro_ws/src/multi_brains/multi_brains/action_service.py
```

- Model service source code:

```
~/M3Pro_ws/src/multi_brains/multi_brains/model_service.py
```

- The main program that implements the robot's visual observation function is the `seewhat` method in the `action_service.py` program:
- The function implementation is: save and display an image from the latest viewpoint.
- Then, it sends a request to the `model_service` node, requesting to provide image feedback to the multi_brains agent in Dify.

```
def seewhat(self):
    """
    Save the current view image and send it as feedback to the Dify agent.
    """
    self.save_single_image()
    msg=LlmRequest()
    msg.llm_request=self.actionlog.get_text("image_feedback")
    msg.robot_feedback=True
    self.llm_request_pub.publish(msg)
    return None

def save_single_image(self):
    """保存图片 / Save a single image"""
    cv_image = self.bridge.imgmsg_to_cv2(self.image_msg, "bgr8")
    cv2.imwrite(self.image_cache_path, cv_image)
    time.sleep(0.05)
    display_thread = threading.Thread(target=self.__display_saved_image)
    display_thread.start()

def __display_saved_image(self):
    """
    显示已保存的图片4秒后关闭窗口 / Display the saved image for 4 seconds before
    closing the window
    """
    try:
        img = cv2.imread(self.image_cache_path)
        if img is not None:
            cv2.imshow("Saved Image", img)
            cv2.waitKey(4000) # 等待4秒 / wait for 4 seconds
            cv2.destroyAllWindows()
        else:
            self.get_logger().error(
                "Failed to load saved image for display."
            ) # Failed to load the saved image for display...
    except Exception as e:
        self.get_logger().error(f"Error displaying image: {e}") # An error
        occurred while displaying the image...
```

- In addition, the `llm_request_callback` function in `model_service.py` is used to receive requests to access the `multi_brains` agent.
- If the `llm_request` field in the request message indicates an image request, a list `[msg.llm_request, 'image_request', True]` is constructed and added to the model request processing queue.

```
def llm_request_callback(self, msg:LlmRequest):
    """
    话题回调函数,接收调用模型请求并放入队列中 / Topic callback function, receive
    model request and put into queue
    """
    if self.debug_mode: self.get_logger().info(f"robot_feedback:
{msg.robot_feedback},llm_request:{msg.llm_request}")
    if msg.robot_feedback:
        # Robot Feedback Request
        if msg.llm_request ==self.syslog.get_text("image_feedback"):

self.llm_handler_queue.put([msg.llm_request,'image_request',True])
        elif msg.llm_request == "finish":
            # Upon receiving the `finish` command from dify-agent, the
            current task cycle ends. This clears the historical context and starts a new task
            cycle.

            self.clear_request_queue() # Clear the request queue.
            self.dify_llmclient.reset_conversation() # Reset session
        else:
            #Conventional robot feedback results
            if self.debug_mode:
self.get_logger().info(self.syslog.get_text("system_log_4"))

self.llm_handler_queue.put([msg.llm_request,'text_request',True])

        else:# Model requests from other sources
            self.llm_handler_queue.put([msg.llm_request,'text_request',None])
```

- In the `handle_llm_thread` method of `model_service.py`, the response mode is determined by the `self.text_chat_mode` variable. If it's only text-based interaction mode, only text responses will be provided.

```
def handle_llm_thread(self)->None:
    """
    处理模型请求/ Handle model request
    """
    while True:
        if not self.llm_handler_queue.empty():#The queue is not empty,
processing model requests.
            request_query = self.llm_handler_queue.get()
            if self.debug_mode: self.get_logger().info(f"Processing LLM
request: {request_query}")

            if request_query[1]=='text_request':
                '''text request'''
```

```

result=self.dify_llmclient.chat(request_query[0],robot_feedback=request_query[2])

        elif request_query[1]=='image_request':
            '''vision + text request'''

result=self.dify_llmclient.chat(request_query[0],image_path=self.image_cache_path,robot_feedback=request_query[2])

        if result[0]:
            if not self.text_chat_mode and self.asr_detect.record_flag: continue

            split_result=self.extract_actions(result[1])
            if split_result is None: continue

action_list,llm_response,decision_plan=self.extract_actions(result[1])
            if decision_plan is not None:

self.get_logger().info(Fore.YELLOW+self.syslog.get_text("system_log_3",decision_plan=decision_plan)+Fore.RESET)
                self.get_logger().info(Fore.YELLOW+f"action":
{action_list},"response": {llm_response}'+Fore.RESET)

                if not self.text_chat_mode:#Voice Reply
                    if
self.tts_engine.synthesize(llm_response,self.tts_out_path) :
                        self.play_audio(self.tts_out_path)
                    else:
                        self.get_logger().error(Fore.RED+"Speech synthesis
failed. Check whether the TTS model is available"+Fore.RESET)
                    else:#Text Reply
                        if decision_plan is not None:

self.text_pub.publish(String(data=self.syslog.get_text("system_log_3",decision_plan=decision_plan)))
                            self.text_pub.publish(String(data=f"action":
{action_list}, "response": {llm_response}'))

                            if action_list!=[]: self.send_action_service(action_list,
llm_response)
                                else:
                                    self.get_logger().error(Fore.RED+f"The model request failed.
Check whether the dify or AI model is normal.\
Error Log:{result[1]}"+Fore.RESET)
                                else:
                                    time.sleep(1.0)#Sleep for 1 second when there are no requests.

```

- When initializing the model engine, the speech recognition and speech synthesis models are loaded only in speech interaction mode.

```

def init_largemodel(self):
    # Initialize Dify

self.dify_llmclient=Dify_LLM_Client(api_key=self.dify_api_key,base_url=self.dify_base_url)

```

```

result=self.dify_llmclient.test_connection()
if result[0]:
    self.get_logger().info("Dify LLM connection successful")
else:
    self.get_logger().error(Fore.RED+f"Dify LLM connection failed,\
Please check whether DIY dify has been
started, API and BASE_URL settings\
Error log:{result[1]}" +Fore.RESET)
    llm_handler_thread = threading.Thread(target=self.handle_llm_thread)#
Start the Dify thread to handle business logic related to large language model
requests.
    llm_handler_thread.daemon = True
    llm_handler_thread.start()

if not self.text_chat_mode: # Voice Chat Mode
    #Initialize ASR
    self.asr_detect=ASR_Detect(self.llm_handler_queue,self.config_file,
pygame_lock=self.pygame_lock)
    if not self.asr_detect.init_ASR_Detect() :
        self.get_logger().error(Fore.RED+"Failed to initialize
ASR_Engine"+Fore.RESET)

    asr_thread =
threading.Thread(target=self.asr_detect.asr_detect_run)#Start the ASR thread to
handle wake-up and speech recognition related tasks.
    asr_thread.daemon = True
    asr_thread.start()

    #Initialize TTS

self.tts_out_path=os.path.join(os.path.expanduser('~'),'M3Pro_ws','multi_brains
_file','tts_output.wav') # 语音合成输出路径 / TTS output path

if self.config_param.get("USE_ONLINE_TTS", False):
    tts_supplier=self.config_param.get("TTS_SUPPLIER")

    if tts_supplier=="aliyun":
        self.tts_engine=TongyiTTS(self.config_file)
        if self.tts_engine.init_tts_engine() :
            self.get_logger().info(Fore.BLUE+"Tongyi TTS_Engine
initialized successfully"+Fore.RESET)
        else:
            self.get_logger().error(Fore.RED+"Tongyi TTS_Engine
initialized failed"+Fore.RESET)
    elif tts_supplier=="baidu":
        self.tts_engine=BaiduTTS(self.config_file)
        if self.tts_engine.init_tts_engine() :
            self.get_logger().info(Fore.BLUE+"Baidu TTS_Engine
initialized successfully"+Fore.RESET)
        else:
            self.get_logger().error(Fore.RED+"Baidu TTS_Engine
initialized failed "+Fore.RESET)

    elif tts_supplier=="xunfei":

self.tts_out_path=os.path.join(os.path.expanduser('~'),'M3Pro_ws','multi_brains
_file','xunfei_tts_output.mp3')

```

```
key_path=os.path.join(os.path.expanduser('~'),self.config_param.get("XUNFEI_KEY", ""))

xunfei_encrypted_key=os.path.join(os.path.expanduser('~'),self.config_param.get("XUNFEI_ENCRYPTED", ""))

self.tts_engine=XunfeiTTS(key_path,xunfei_encrypted_key,self.config_file)

        if self.tts_engine.init_tts_engine():
            self.get_logger().info(Fore.BLUE+"Xunfei TTS_Engine
initialized successfully"+Fore.RESET)
        else:
            self.get_logger().error(Fore.RED+"Xunfei TTS_Engine
initialized failed"+Fore.RESET)

    else:
        self.tts_engine=PiperTTS(self.language)
        if self.tts_engine.init_tts_engine() is not False:
            self.get_logger().info(Fore.BLUE+"Piper TTS_Engine
initialized successfully"+Fore.RESET)
        else:
            self.get_logger().error(Fore.RED+"Piper TTS_Engine
initialized failed"+Fore.RESET)
```